

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA0302

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Code of Conduct:

I __M Poojitha Reddy- 192472352__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding ; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and	Provides reasonable analysis with some	Limited analysis; weak prioritization	No meaningful analysis or

		prioritizes them logically	prioritization		prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

- 1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.**

CODE :

```
import gymnasium as gym
from gymnasium import spaces
import numpy as np
import random

class GridWorld(gym.Env):
    def __init__(self):
        self.size = 5
        self.start = 0
        self.goal = 24
        self.obstacles = [6, 13, 17]
        self.observation_space = spaces.Discrete(25)
        self.action_space = spaces.Discrete(4)

    def reset(self, seed=None, options=None):
        self.state = self.start
        return self.state, {}

    def step(self, action):
        r, c = divmod(self.state, 5)

        if action == 0: r -= 1
        if action == 1: r += 1
        if action == 2: c -= 1
```

```

        if action == 3: c += 1

        new = r * 5 + c

        if r < 0 or r >= 5 or c < 0 or c >= 5:

            return self.state, -1, False, False, {}

        self.state = new

        if self.state in self.obstacles:

            return self.state, -10, True, False, {}

        if self.state == self.goal:

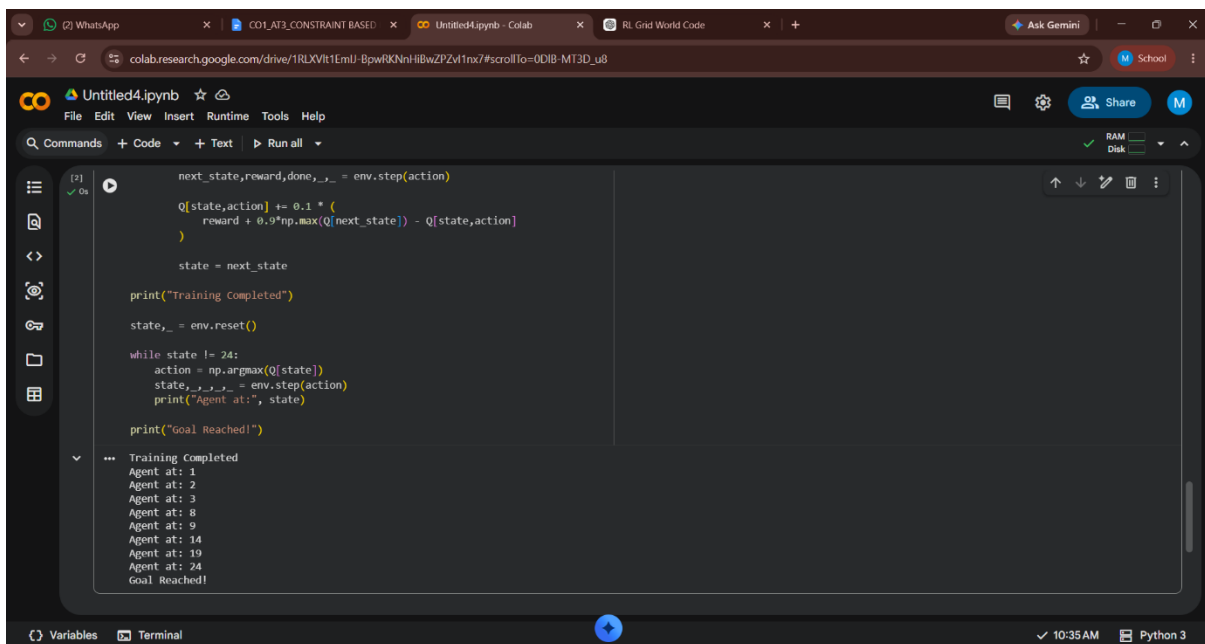
            return self.state, 10, True, False, {}

        return self.state, -1, False, False, {}

env = GridWorld()
Q = np.zeros((25,4))
for episode in range(500):
    state,_ = env.reset()
    done = False
    while not done:
        action = random.randint(0,3) if random.random()<0.1 else np.argmax(Q[state])
        next_state,reward,done,_,_ = env.step(action)
        Q[state,action] += 0.1 * (
            reward + 0.9*np.max(Q[next_state]) - Q[state,action]
        )
        state = next_state
    print("Training Completed")
    state,_ = env.reset()
    while state != 24:
        action = np.argmax(Q[state])
        state,_,_,_ = env.step(action)
        print("Agent at:", state)
    print("Goal Reached!")

```

OUTPUT :



```
next_state, reward, done, _ = env.step(action)

Q[state, action] += 0.1 * (
    reward + 0.9 * np.max(Q[next_state]) - Q[state, action]
)

state = next_state

print("Training Completed")

state, _ = env.reset()

while state != 24:
    action = np.argmax(Q[state])
    state, _ = env.step(action)
    print("Agent at:", state)

print("Goal Reached!")

... Training Completed
Agent at: 1
Agent at: 2
Agent at: 3
Agent at: 8
Agent at: 9
Agent at: 14
Agent at: 19
Agent at: 24
Goal Reached!
```

2. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

CODE :

```
import numpy as np

states = ["Start", "A", "B", "Goal"]

actions = ["Move", "Charge"]

P = {
    "Start": {"Move": "A"},
    "A": {"Move": "B", "Charge": "A"},
    "B": {"Move": "Goal", "Charge": "B"}
}

R = {
    "Start": -2,
    "A": -3,
    "B": -2,
    "Goal": 10
}

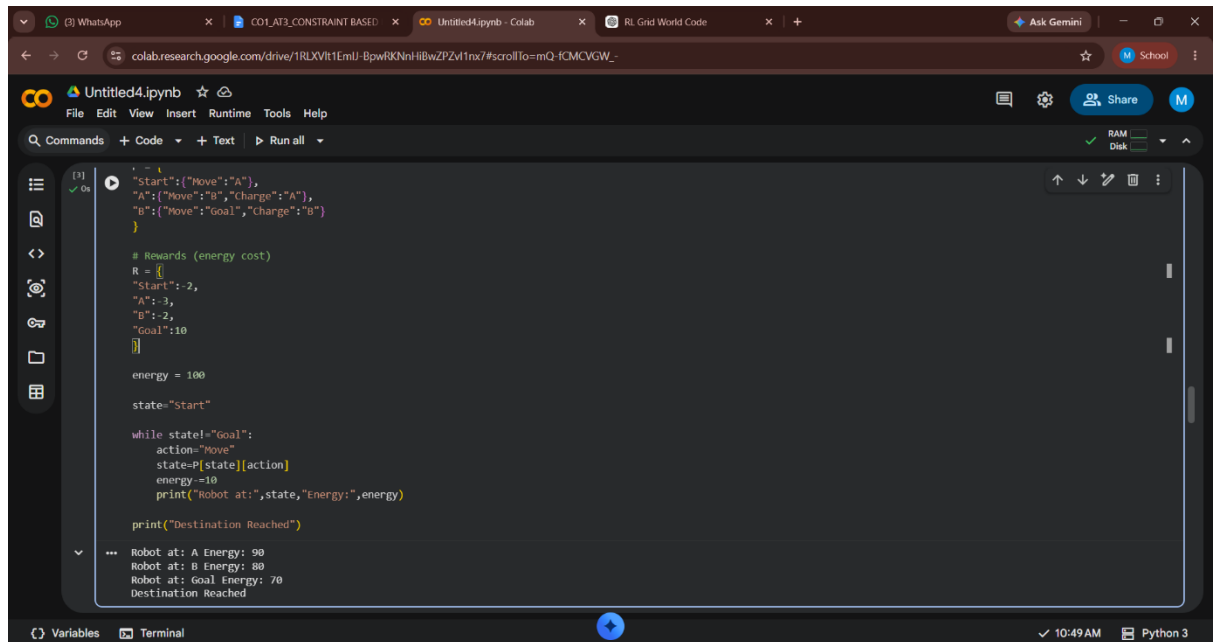
energy = 100
state = "Start"
```

```

while state!="Goal":
    action="Move"
    state=P[state][action]
    energy-=10
    print("Robot at:",state,"Energy:",energy)
print("Destination Reached")

```

OUTPUT :



The screenshot shows a Google Colab notebook titled 'Untitled4.ipynb'. The code cell contains a Python script that simulates a robot's movement between states 'Start', 'A', 'B', and 'Goal'. The robot starts at 'Start' with 100 energy. It moves to 'A' (energy 90), then to 'B' (energy 80), and finally to 'Goal' (energy 70). The output of the script is displayed below the code cell.

```

{
  "Start": {"Move": "A"},
  "A": {"Move": "B", "Charge": "A"},
  "B": {"Move": "Goal", "Charge": "B"}
}

# Rewards (energy cost)
R = {
  "Start": -2,
  "A": -3,
  "B": -2,
  "Goal": 10
}

energy = 100
state = "Start"

while state != "Goal":
    action = "Move"
    state = P[state][action]
    energy -= 10
    print("Robot at:", state, "Energy:", energy)

print("Destination Reached")

```

Robot at: A Energy: 90
 Robot at: B Energy: 80
 Robot at: Goal Energy: 70
 Destination Reached

3. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

CODE :

```

import random

import numpy as np

deliveries = 0

battery = 30

Q = np.zeros((31,2)) # state = battery, actions = deliver/recharge

for episode in range(100):

    battery = 30

    while battery > 0:

```

```

action = random.choice([0,1])

if action == 0:

    reward = 10

    deliveries += 1

    battery -= 5

else:

    reward = -2

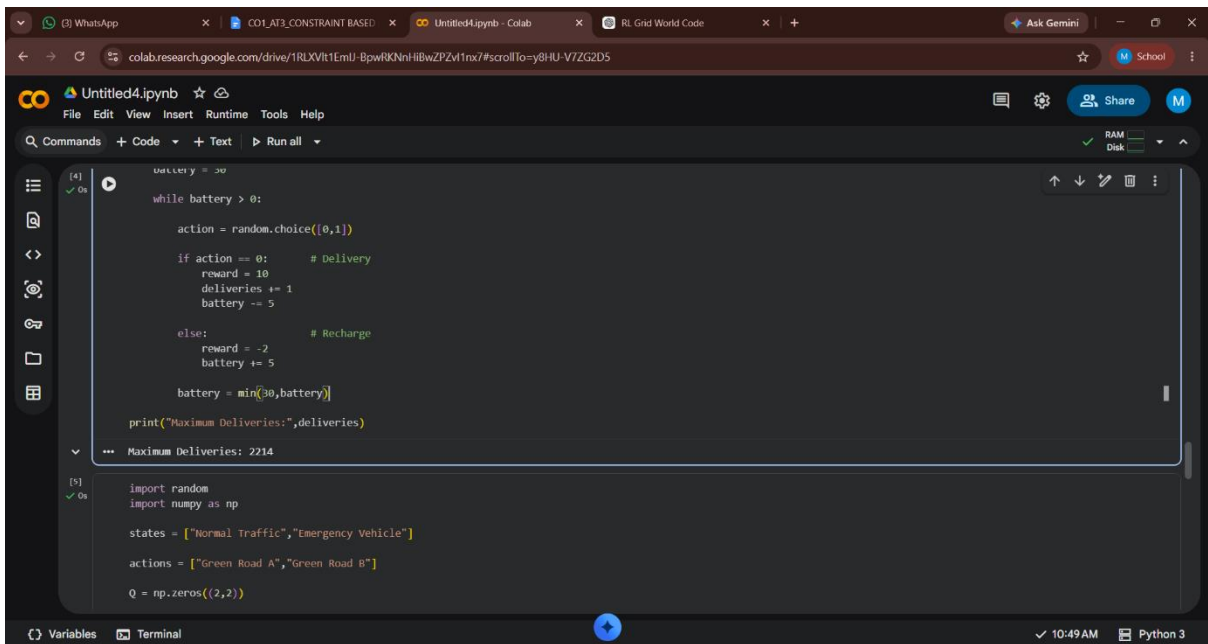
    battery += 5

battery = min(30,battery)

print("Maximum Deliveries:",deliveries)

```

OUTPUT :



```

battery = 30
while battery > 0:

    action = random.choice([0,1])

    if action == 0: # Delivery
        reward = 10
        deliveries += 1
        battery -= 5

    else: # Recharge
        reward = -2
        battery += 5

    battery = min(30,battery)

print("Maximum Deliveries:",deliveries)

--- Maximum Deliveries: 2214

```

```

import random
import numpy as np

states = ["Normal Traffic","Emergency Vehicle"]

actions = ["Green Road A","Green Road B"]

Q = np.zeros((2,2))

```

4. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

CODE :

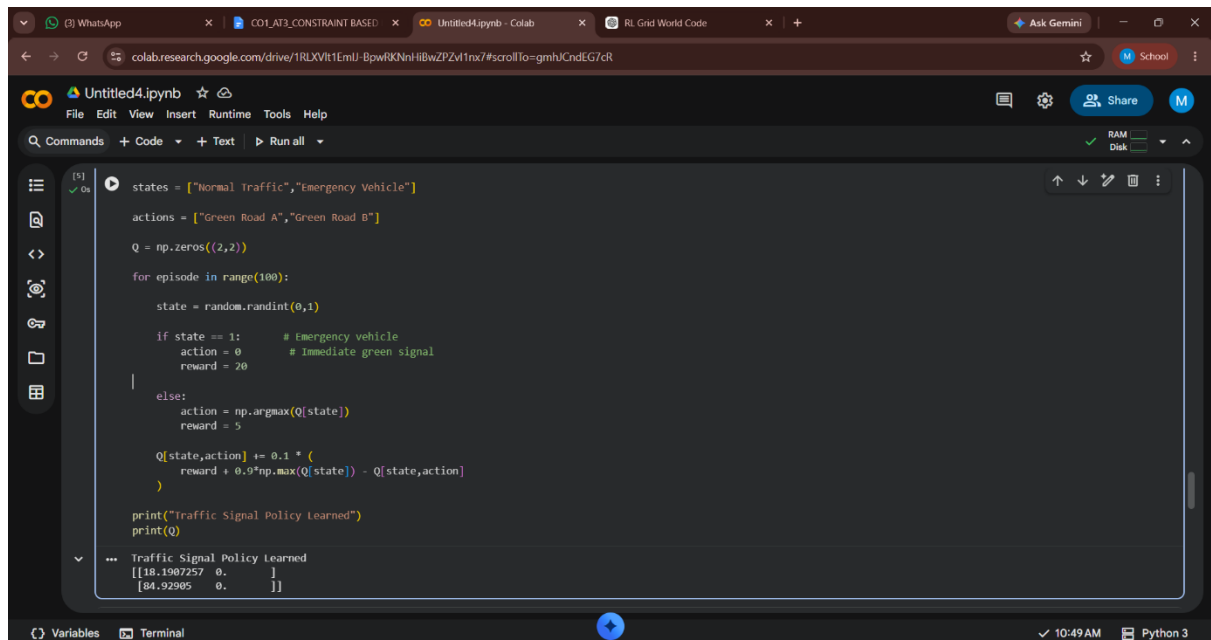
```

import random
import numpy as np
states = ["Normal Traffic", "Emergency Vehicle"]
actions = ["Green Road A", "Green Road B"]
Q = np.zeros((2,2))
for episode in range(100):
    state = random.randint(0,1)
    if state == 1:
        action = 0
        reward = 20
    else:
        action = np.argmax(Q[state])
        reward = 5
    Q[state,action] += 0.1 * (
        reward + 0.9*np.max(Q[state]) - Q[state,action]
    )

print("Traffic Signal Policy Learned")
print(Q)

```

OUTPUT :



```

states = ["Normal Traffic", "Emergency Vehicle"]
actions = ["Green Road A", "Green Road B"]
Q = np.zeros((2,2))

for episode in range(100):
    state = random.randint(0,1)

    if state == 1: # Emergency vehicle
        action = 0 # Immediate green signal
        reward = 20
    else:
        action = np.argmax(Q[state])
        reward = 5

    Q[state,action] += 0.1 * (
        reward + 0.9*np.max(Q[state]) - Q[state,action]
    )

print("Traffic Signal Policy Learned")
print(Q)

```

... Traffic Signal Policy Learned

```

[[18.1907257  0.        ]
 [84.92905    0.        ]]

```

5. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

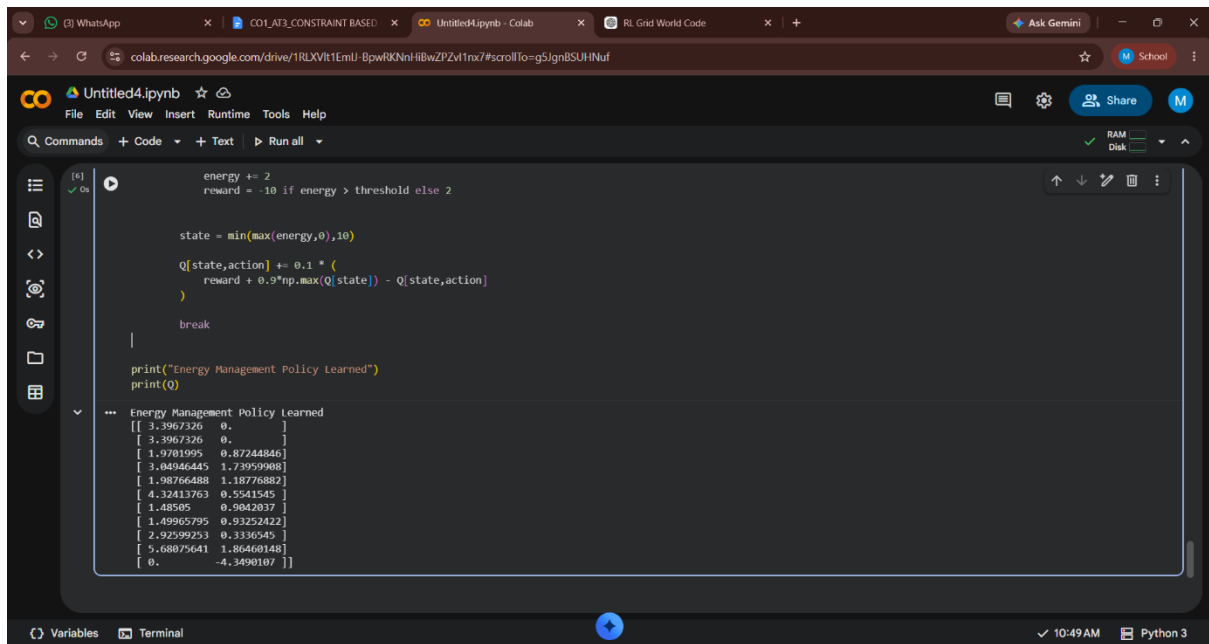
CODE :


```

import numpy as np
import random
Q = np.zeros((11,2))
threshold = 10
for episode in range(100):
    energy = random.randint(0,10)
    while energy <= threshold:
        action = random.choice([0,1])
        if action == 0:
            energy -= 1
            reward = 5
        else:
            energy += 2
            reward = -10 if energy > threshold else 2
        state = min(max(energy,0),10)
        Q[state,action] += 0.1 * (
            reward + 0.9*np.max(Q[state]) - Q[state,action]
        )
        break
    print("Energy Management Policy Learned")
print(Q)

```

OUTPUT :



```
energy += 2
reward = -10 if energy > threshold else 2

state = min(max(energy,0),10)

Q[state,action] += 0.1 * (
    reward + 0.9*np.max(Q[state]) - Q[state,action]
)

break

print("Energy Management Policy Learned")
print(Q)
```

Energy Management Policy Learned

[3.3967326	0.
[3.3967326	0.
[1.9701995	0.87244846]
[3.04946445	1.73959908]
[1.98766488	1.18776882]
[4.32413763	0.5541545]
[1.48505	0.9842037]
[1.49057895	0.93232422]
[2.92599253	0.3336545]
[5.68075641	1.86460148]
[0.	-4.3490107]]

GitHub link : <https://github.com/Poojithareddy1092/RL>